

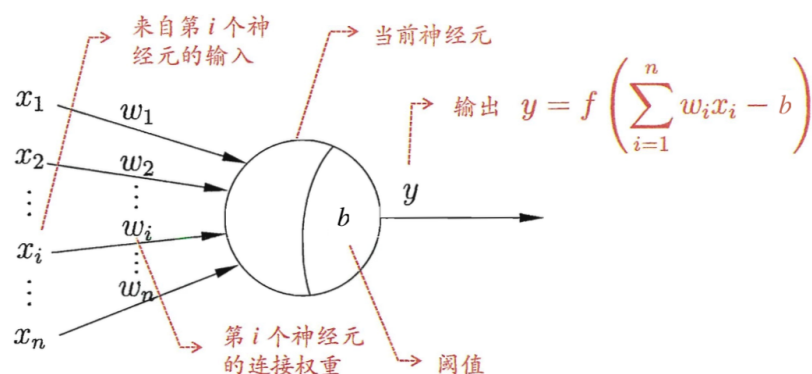
教AI的陶老师作业训练营第4期（2）

一、理论部分

有了机器学习的基础知识，这一周咱们进入神经网络的学习。

神经网络？不是深度学习么？①二者是什么关系？

随着人们对自然的认识，仿生学也逐渐发展起来，我们要学习的人工神经网络就是这其中一个产物。1943年，McCulloch与Pitts从对人脑神经元的认识抽象出下图所示的简单模型，这就是著名的“M-P神经元模型”。在这个模型中，神经元接收到来自 n 个其它神经元传递过来的输入信号，这些输入信号通过带权重的连接对输入信号进行线性变换，神经元接收到的总输入与神经元的阈值进行比较，然后通过激活函数处理以产生神经元的输出。



注意：上述模型中 $x_1, x_2, \dots, x_n$ 表示外部刺激，也可记为 $\mathbf{x}$ ，也就是模型输入。

$w_1, w_2, \dots, w_n$ 为突触权值，也就是模型自由参数，也可记为 $\mathbf{w}$ 。 b 表示阈值。 y 代表计算后的输出。函数 f 我们下面细说。

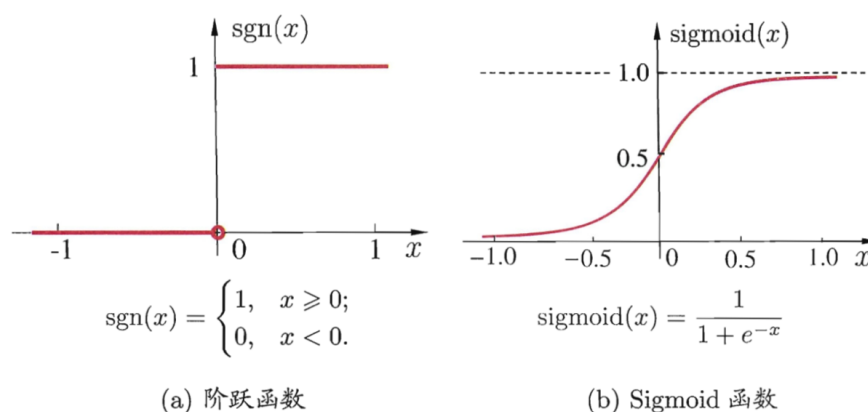
根据上图请尝试回答：

②为什么说M-P神经元将所有信号的输入看做是同时发生的。

③输入的信号在传导的过程中分别做了怎样的计算。

假设人脑神经元有兴奋和抑制两种状态，为了模拟类似效果，现在希望输入信息在经过神经元之后会得到两种输出，“1”和“0”，分别代表神经元兴奋和神经元抑制。而这里的函数 f 的作用就是来完成这件事，这就是我们常说的**激活函数**，如下图左图所示。

但现在这个激活函数具有不连续、不光滑等不太好的性质，因此实际中常常使用Sigmoid函数作为激活函数，其中常用的一种Sigmoid激活为Logistic激活函数如下图右图所示。



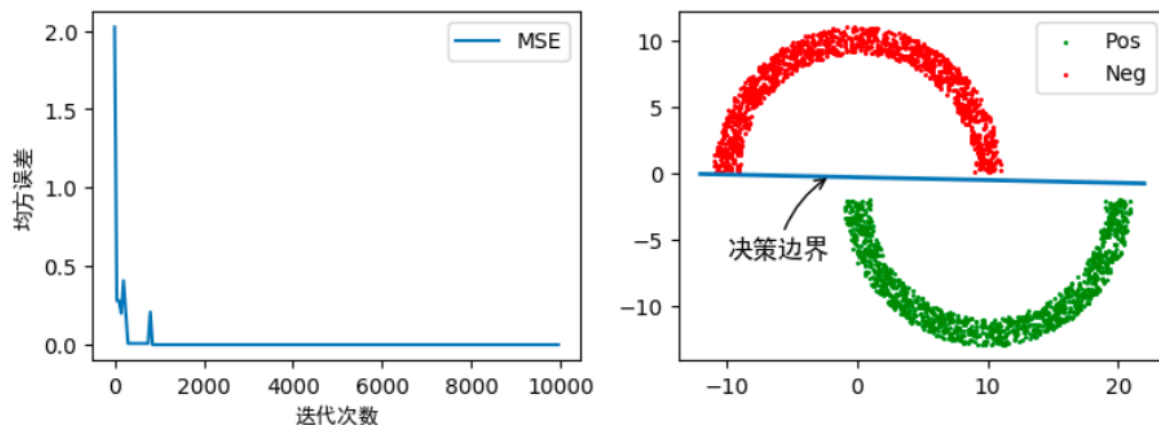
④常见到的激活函数还有Relu与leakyRelu, tanh等, 请写出各自函数定义。

⑤在Python环境下使用Numpy编写Sgn、Logistic激活函数。

说到这里, 不得不提一下大名鼎鼎的Rosenblatt感知器。在McCulloch and Pitts (1943) 引入神经网络的概念, 并设计了M-P模型后, Rosenblatt (1958) 提出的感知器则作为神经网络中监督学习的第一个模型。它在M-P模型的基础上添加了训练方法, 并且Rosenblatt证明了当用来训练感知器的模式(向量)取自两个线性可分的类时, 感知器算法是收敛的。算法的收敛性证明被称为感知器收敛定理(如果你有兴趣且数学不错, 可以去看看这个证明过程)。

⑥感知器的收敛算法是怎样的? 请用自己语言描述。(选做)

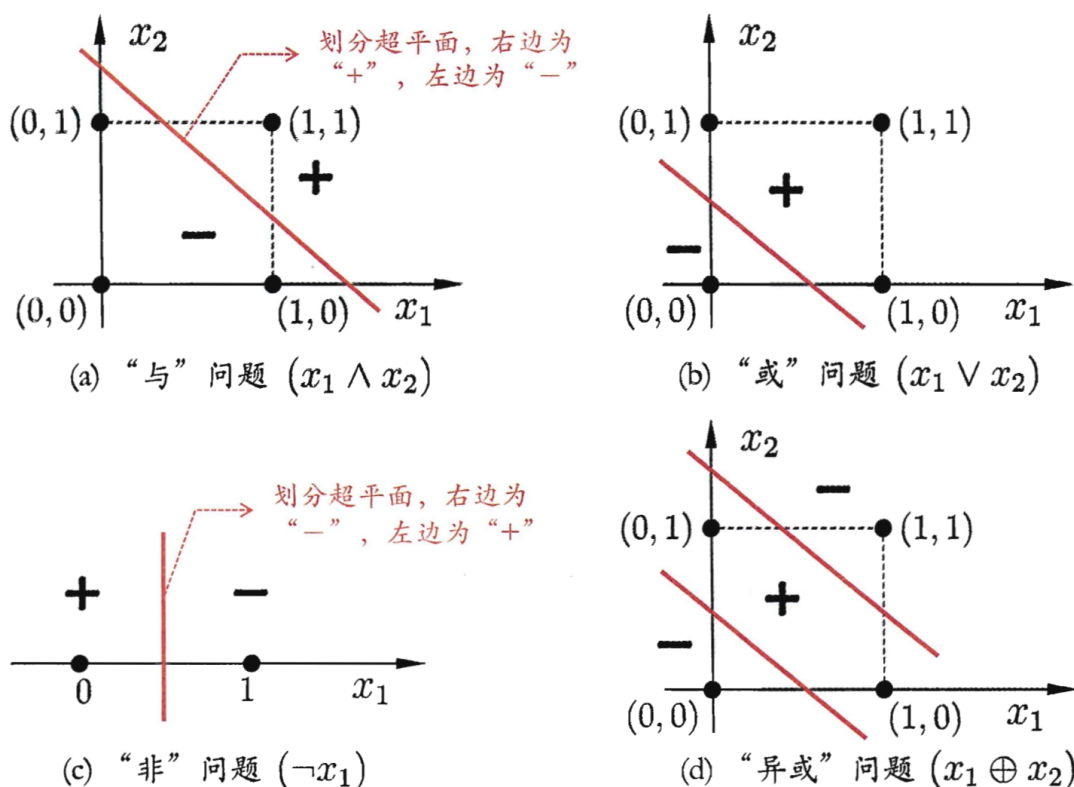
⑦代码实现Rosenblatt感知器与收敛算法, 并使用均方误差(MSE) 衡量模型性能。使用线性可分的双月数据集进行训练与测试。(选做) 参考效果如图:



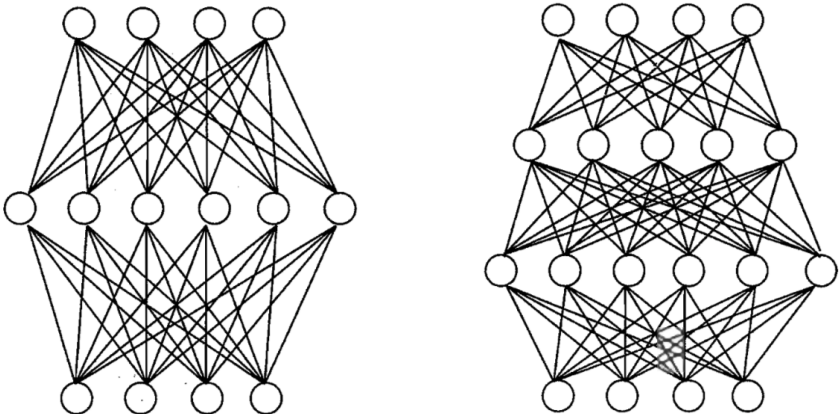
提示: scikit-learn中有直接生成双月数据集的函数方法。

感知器现在已经没有人使用了, 但如果你能将问题⑦实现, 不光是无疑的学霸, 更能给你的编程能力带来提升。

以上都只是一个神经元, 还不能称之为网络。(但是复杂的神经网络也就是由这种结构的众多神经元相互连接构成。) 更重要的是, 一个神经元无法完成像抑或问题这样线性不可分的复杂问题。如下图:



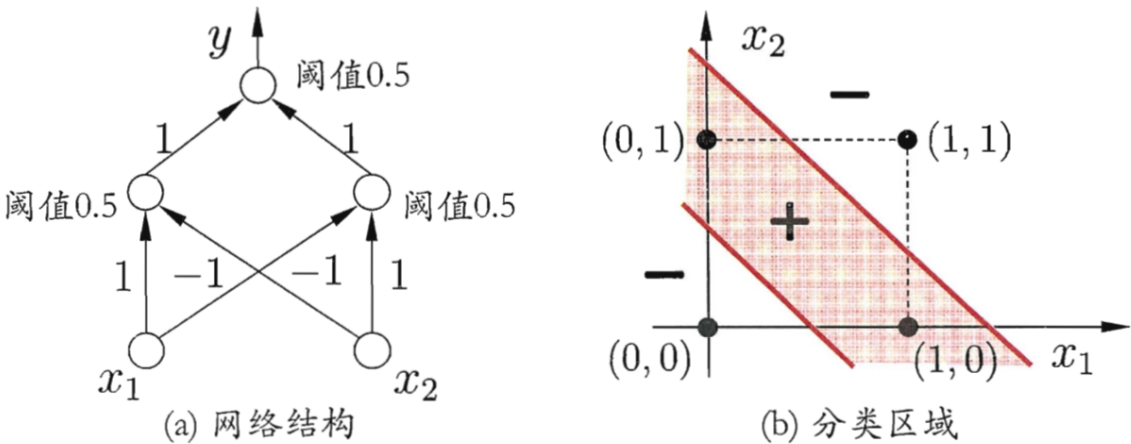
在实际中遇到的大多数模式分类问题都是线性不可分问题，这就使得感知器的用途变得极为狭窄。要解决线性不可分问题，需要使用多层功能神经元（多于0个隐藏层）。简单的来看就是把许多个神经元排成一排，可以组成一个单层前馈神经网络，这个网络的每一个神经元接收相同的数据作为输入，但每个神经元具有不同的连接权重、阈值，这样的一层网络可以输出与神经元一样多数量的值。单层前馈神经网络的输出也可以作为另一个单层神经网络的输入，以此相连在一起就组成了规模庞大的神经网络。



那么，⑧什么是输入层、隐藏层和输出层？

注意：每个层的人工神经元与前一层神经元全连接，即为全连接神经网络（也存在其它连接形式，如卷积神经网络的局部连接等）。

上述异或问题只需要三层全连接的神经网络（包含一个隐藏层的神经网络）即可解决。其中隐藏层与输出层均包含有非线性的激活函数。如下图所示，左图为3层全连接神经网络的结构图，图中标明了连接权重与阈值；右图表示对应的决策边界与决策域。



这里每一个神经元模型为M-P模型，所以其激活函数为：

$$\text{sgn}(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

⑨请将上面右图(b)分类区域中4个点分别按左图(a)网络参数设置进行计算。

现在你应当清楚的体会到，模型的节点（神经元）数量越多，参数越大，模型的拟合能力也就越强了。那么请你思考，⑩激活函数的作用到底是什么？如果没有激活函数网络会怎样？

在你完成第⑨问的过程中，就像是拿一个已经训练好的模型去进行预测。这里的参数（**w**，**b**）是已知的，你只是将数据输入在这个网络结构里从前往后的进行了计算，最终获得预测结果。当然你现在计算的结果和右图情况是相符的，因为现在你有一个能够很好拟合当前问题的模型。得到这个能够拟合你要解决的问题的模型，就是我们进行模型训练的目的。当模型结构已定时，能不能拟合要解决的问题，就取决于模型里面参数的取值。所以说人话就是，训练模型的目的，就是去寻找当前结构下的最优参数。

上次作业讲到训练模型前首先确定三件事：1. 选择一个具有未知参数的模型；2. 定义损失函数；3. 确定优化方法。

在这基础上，就可以通过“前向传播(求得损失)-->反向传播(更新参数)"的循环进行来训练模型了。

求得损失需要通过调用损失函数来进行。损失函数的目的就是看看现在模型的预测情况和真实值的差距有多大（有监督情况下）。⑪对于回归问题、二分类问题、多分类问题通常使用的损失函数有哪些？

下面就该重头戏”反向传播算法“出场了。

反向传播（Backpropagation，BP）是“误差反向传播”的简称，是一种与最优化方法（如梯度下降法）结合使用的，用来训练人工神经网络的常见方法。该方法对网络中所有权重计算损失函数的梯度。这个梯度会反馈给最优化方法，用来更新权值以最小化损失函数。

人话翻译：在前向传播获得预测之后，就可以通过损失函数求得损失。求得损失之后，我们希望能够对参数进行调整，使得损失能够在当前情况下变小一点，也就是往让损失减小的方向调整一次参数。这个让损失减小的方向，就可以通过求得梯度来解决。更新的基本方法为：对于一个参数 w ，求得它的梯度 d 后，让它往梯度的反方向以指定步长 a （学习率）走一步。也就是 $w(\text{new}) = w - a * d$ ，这里的 a 是超参数。

注意！根本之处在于：损失本来就是一个关于参数 w 和 b 的函数！

作为老师的我，这里真想给大家画上几个黑板说的清清楚楚，但形式受限，不如就⑫请你来手撕画图将梯度下降过程表述清楚。

学到这里，你应该已经清楚了神经网络的训练过程。下一步的问题就是搞定梯度值。

高数那里讲的东西咱们这里就不再细说，我只想说，为了求导方便，为了计算机计算方便，这里使用链式法则来进行求导，也就是会从损失反向往前求得参数节点的导数。

⑬看我的视频”手撕神经网络前向、反向、求导、更参“，将例子中的9个参数分别手撕求导，并更新参数。

⑭梯度下降、随机梯度下降与小批量梯度下降都分别是怎样设计的？

二、编程部分

这次我们开始说PyTorch。

通常对于PyTorch的导入写成这样：

```
import torch
```

PyTorch中的数据类型是tensor。类似NumPy中ndarray，唯一区别是tensor可以在GPU上运算。

⑮请你构建一个三行四列由随机数填充的tensor类型变量，并通过属性获得它的形状。

⑯ndarray和tensor两种类型如何进行相互转换。

下面代码实现相关问题

```
In [2]: import torch

x = torch.tensor([[1., 0.],[-1., 1.]],
                  requires_grad = True)
z = x.pow(2).sum()
z.backward()
x.grad
```

```
Out[2]: tensor([[ 2.,  0.],
               [-2.,  2.]])
```

⑰这四行代码都用了哪些函数或属性，具有什么作用？请进行详细解释。

⑱如果将 `requires_grad = True` 设置为 `requires_grad = False` 会怎样？这个属性的作用是什么？

⑲ `z.backward(.)` 执行后，都发生了什么？

⑳将我的视频”手撕神经网络前向、反向、求导、更参“例子对应代码实现。

最后，有余力的同学去看看PyTorch中的计算图是怎么回事，这会让你对自动微分这件事儿更通透，也会让你在网络的搭建过程中更加明朗。